



NUS Orbital 2026

Project Group:

F4C_Opossum_6748

Prepared by:

WONG BO XI FILBERT

LI JIANXI

Table of Contents

1. Executive Summary	3
2. Project Vision & Motivation	3
2.1. Mission Statement	3
2.2. Academic Motivation	3
3. Target Audience & User Stories	3
4. Core Game Features (Milestone 1)	4
4.1. Basic 2D Engine setup	4
4.2. Entity Spawning System	4
4.3. Combat & Movement AI	6
4.4. Modular Health & UI System	8
4.5. Dynamic Camera System	8
5. Future Roadmap (Milestones 2 & 3)	9
5.1. Milestone 2 Priorities	9
5.2. Milestone 3 Priorities	10
6. Modern Technical Stack	10
7. Software Engineering Rigour	10
8. Methodology & Quality Assurance	11
9. Conclusion for Milestone 1	11

1. Executive Summary

This document serves as the official Milestone 1 formal report for Project F4C Opossum, developed for the NUS Orbital 2026 programme. Aiming for the Apollo 11 level of achievement, our team is engineering a highly scalable, robust 2D side-scrolling strategy game using the Unity Engine and C#. This first milestone demonstrates a fully functional Proof of Concept (PoC) that encompasses the core game loop: resource economy, automated entity spawning, AI-driven adversarial generation, collision-based combat, and a definitive win/loss state via base destruction.

2. Project Vision & Motivation

2.1. Mission Statement

Inspired by the timeless mechanics of the classic flash game Age of War, F4C Opossum modernises the tug-of-war base defence genre. The game challenges players to balance a constantly ticking resource economy with tactical unit deployment to overwhelm a dynamic enemy AI.

2.2. Academic Motivation

As Computer Science undergraduates, we see this project as a deliberate push beyond standard university curricula. Building a game from scratch requires mastering complex, real-time software architectures, UI/UX interaction paradigms, and optimising intricate game loops. Targeting Apollo 11 requires us not only to deliver a fun game but also to adhere strictly to industry-standard software engineering practices, ensuring our codebase is clean, modular, and extensible.

3. Target Audience & User Stories

To ensure our development remains user-centric, we have defined four core player personas:

- **The Strategist:** "As a strategist, I want to balance my gold economy between spawning cheap frontline units and saving for expensive ranged units, so I can optimally counter the enemy's formation."
- **The Competitor:** "As a competitive player, I want the enemy AI to spawn units autonomously and unpredictably, so the game remains challenging and forces me to adapt my strategy on the fly."
- **The Explorer:** "As an explorer, I want to see visual feedback when my units clash with the enemy, so I can clearly understand the combat mechanics and health systems."

- **The Casual Gamer:** "As a casual player, I want an intuitive user interface with clear buttons and resource counters, so I can understand how to play within the first ten seconds of launching the game."

4. Core Game Features (Milestone 1)

To build a solid foundation for F4C Opossum, our initial milestones focused on establishing the core gameplay loop, entity interactions, and robust C# scripts. Currently, our Milestone 1 submission focuses on building the technical foundation necessary for the future gameplay systems. The current prototype demonstrates:

- A functioning 2D environment
- Unit spawning systems
- Movement and collision systems
- Base health management
- Combat interactions
- Dynamic HP bar rendering

These features help to establish the minimum viable gameplay loop required for future expansion.

4.1. Basic 2D Engine setup

To start the project, we had to set up the overview of how the gamemap would look, as well as the layers we had to consider when building the map. For example, different layers like (foreground, default, background), as well as layers for unit tagging to differentiate between allied units and enemy units.

As we are planning to build an Age of War game based on a 2D side-scrolling game, we decided to utilise the Unity Universal 2D template.

4.2. Entity Spawning System

Since Age of War is a 2D Player versus Enemy game that requires user input to spawn units against AI/bot enemies, we needed to create spawning scripts for both player-controlled and enemy-controlled units.

Current Functionality:

- Dynamic runtime instantiation
- Separate spawn points
- Independent player/enemy spawning systems

Scripts Created for Spawning System:

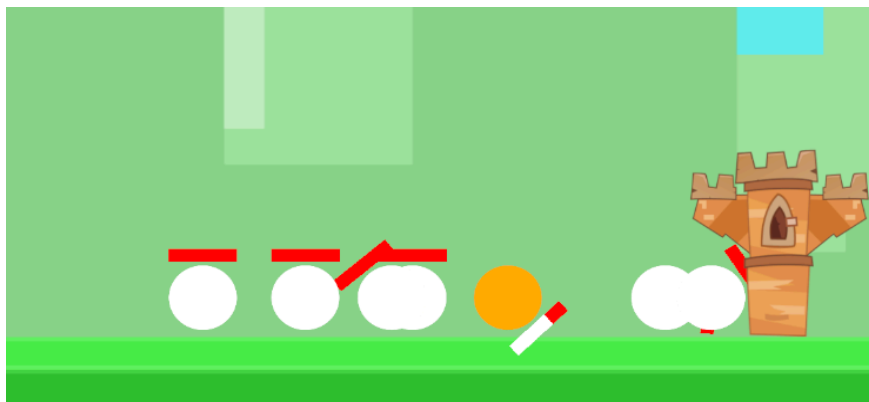
- PlayerSpawner.cs
- EnemySpawner.cs

Design Configurations:

- Player unit prefabs are loaded dynamically from a "Resources/PlayerUnits" folder at startup.
- The player (user) can deploy units by pressing the "Z" key.
- Originally, a 0.5-second cooldown was enforced on player spawning to prevent units from jittering down and improperly stacking. However, since we had factored in the idea that the unit stacking is alright for allies only at the base, the original jittering issue has been resolved, where the cooldown now is to simulate the time it takes to "train" up the unit.
- Enemy units are similarly loaded from a "Resources/EnemyUnits" folder.
- Enemies currently spawn automatically on a 25-second timer.
- Both spawners actively monitor the battlefield and will halt all spawning if their respective base's HealthSystem is destroyed.
- A sequential spawnOrder tracker tags every instantiated unit, allowing the game to distinguish older units from newer ones. This feature prevents unit stacking bugs, as stacked units now know which unit should move first.

Challenges faced during the building process of this feature:

One issue encountered was unit overlap during spawning. This occasionally caused units to collide incorrectly or become stuck during movement. This was partially resolved by implementing collision checking and movement spacing between allied units. The image below shows the units colliding during testing of the collision mechanics. In the previous version, the enemy unit spawn was too close to the map, causing enemy units to spawn on "ally" units and creating this undesired bug.



Design Decisions and Future Considerations:

We chose to create two separate scripts, one for ally spawning and one for enemy spawning, since we know that enemy spawning is initiated by the Computer code, so there is no need to add features that read user input to spawn units. On the other hand, since the player units are spawned not based on time but by user input, we need to factor the reading of user inputs into the PlayerSpawner.cs script.

However, future considerations include adding features like monetary tracking so that units can only be spawned when the player/enemy has enough money to deploy them, as well as creating buttons to deploy units instead of using keycap values for players. That way, the user can see what the unit they are deploying looks like before choosing to deploy it.

4.3. Combat & Movement AI

The combat and Movement feature allows units to move automatically toward enemy targets and engage in combat once enemies enter range. The units are also able to do smart switching between enemy units and the enemy base, where they will switch targets to the earliest spawned unit to attack if they were initially attacking the enemy base when this enemy unit was spawned.

Current Functionality:

- Automatic movement
- Enemy detection
- Attack cooldown systems
- Damage application
- Base destruction logic

Scripts Created for Unit Movement and HP interactions:

- UnitMove.cs
- HealthSystem.cs

How the Combat System works:

- Units begin in an undeployed state and must fall to physically hit the ground layer (detected via Physics2D.OverlapCircle) before they can begin marching.
- Movement direction is determined dynamically by Unity tags; players move right (positive X direction), while enemies move left (negative X direction).

- **Ally Collision:** Units utilise 2D Raycasts (Physics2D.RaycastAll) filtered by layer masks to detect allies directly ahead of them. A unit will stop and wait if the ally in front of it has an older spawnOrder, effectively preventing units from clipping through or passing each other.
- **Targeting:** Units use dynamic bounding boxes (Physics2D.OverlapBoxAll) to detect targets within their specific attack range. The AI scans the overlapping zone and prioritises attacking the oldest spawned enemy unit first. If no enemy units are alive, the AI seamlessly switches its target to the enemy base.
- Combat operates on a 1-second attack cooldown. (This is a placeholder value which can be customised for different units in the future)
- Units deal their predefined damage values directly to the target's HealthSystem once the cooldown completes.

Design Decisions and Future Considerations:

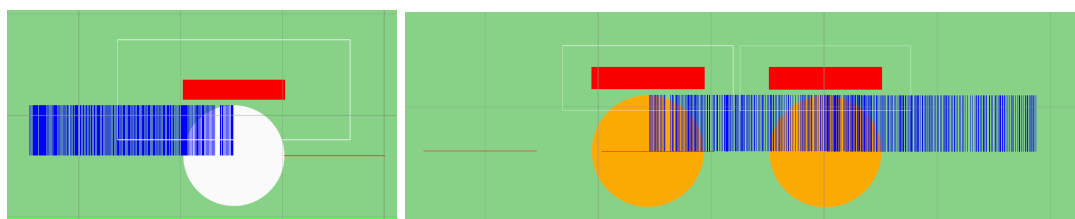
The current combat system design has multiple functions that handle the following:

- Movement logic
- Health management
- Attack behaviour

However, all these functions are currently in a single file, UnitMove.cs. As such, future considerations include breaking these functions into new files to keep them clean and more meaningful.

Challenges faced during the building process of this feature:

Throughout the process of creating, there were a lot of trials and errors in fine-tuning the unit collision detection for both allies (RayCast) and enemies (BoxAll) to ensure no unintended behaviour would occur during gameplay testing. The image below shows the debugging steps and trackers we place to see how the combat collision detection system is used, with the red line indicating the unit's "sight" in front to determine whether the unit in front is an ally or a foe. The blue line is all objects it was colliding with, which we found out was because it was tracking the collision with itself as well.



4.4. Modular Health & UI System

Following a Dynamic HP bars template from YouTube tutorials, we implemented it for both bases and combat units. We also fine-tuned the template, allowing the Health Points bar to drop vertically or horizontally so the system accommodates both orientations.

Current Functionality:

- Real-time HP updates
- World-space UI tracking
- Visual combat feedback

Scripts Created for Health Bar UI:

- HealthBarUI.cs
- HealthSystem.cs

How the Health and UI System works:

- We developed a modular HealthSystem component to manage hit points, with a default maximum of 100 HP (temporary value). Currently, bases have 500 HP, enemy units have 100 HP, and ally units have 150 HP.
- The system takes damage, clamps health values to prevent bugs, and automatically destroys the GameObject when HP reaches zero.
- The system is decoupled from the HealthBarUI script, which visually represents health percentages by adjusting a RectTransform's size.
- The UI supports horizontal orientations for standard units and vertical orientations for the bases.

Challenges faced during the building process of this feature:

Initially, a challenge was how to get the HealthBar to move with the units rather than remain static. since the tutorial did not really cover assigning the health bar to a unit. After searching for a bit, the fix was to put the health bar, in this case the “Canvas UI”, as a child of the UI; that way, when the unit moves, the health bar moves along with the unit.

4.5. Dynamic Camera System

Originally, we created the Gamemap layout to fit only the current camera point of View, with the entire map situated nicely within the camera boundaries. However, during testing, we felt that the “runway” for units to move was too short. As a result, to expand the map, we also need to ensure that the camera can move

freely across the map without needing to be moved by following a player unit (as covered in the Mission Control 2 Webinar), but by following the user's mouse cursor instead.

Therefore, this feature is designed to let a camera controller keep the battlefield visible throughout gameplay, allowing the user to see different parts of the map.

Scripts Created for Health Bar UI:

- CameraController.cs

How the Camera Controller movement works:

- Players can pan the camera horizontally by moving the mouse cursor within 50 pixels of the edges of the screen.
- The camera's movement is clamped to predefined background boundaries to keep the focus on the battlefield.
- Users can utilise the mouse scroll wheel to zoom in and out.
- The zoom, which utilises a mouse scroll wheel, is clamped between a minimum and maximum value, and the camera is mathematically anchored to ground level, so zooming out reveals more of the map rather than exposing underground areas.

5. Future Roadmap (Milestones 2 & 3)

With the MVP architecture secured, development will now pivot to expanding complexity, balance, and visual fidelity.

5.1. Milestone 2 Priorities

- **Monetary Systems with Buttons to Spawn Units:** Implement a currency tracker that tracks the player's current balance to identify which units the player can summon and which units the player is unable to summon. This includes earning money by defeating enemy troops, having a base starting amount, and potentially having a "turret" that, instead of attacking enemy units, helps earn "coins."
- **Art Integration:** Replace placeholder geometry with fully animated 2D sprites (idle, walk, attack, and death animations).
- **Implementation of Turret Defence:** Since our Bases right now are quite empty, with nothing to protect them, we need to implement slots that can be unlocked via the currency, as well as turrets that can be placed on the unlocked slots to

deal damage to enemy when within their range or do passive abilities, like gaining currency.

5.2. Milestone 3 Priorities

- **Special Abilities:** Integrate cooldown-based global abilities (e.g., calling down a meteor strike on the battlefield).
- **Advanced AI:** Upgrade the EnemySpawner script to read the player's unit composition and intelligently spawn counter-units, rather than relying on randomised timers.
- **Audio Engineering:** Add background tracks, UI interaction clicks, and combat sound effects.
- **Polish & CI/CD:** Finalise game balance, conduct extensive bug testing, and potentially set up automated web-GL deployment via GitHub Actions.

6. Modern Technical Stack

F4C Opossum was built using industry-standard tools to ensure a high-quality end product.

- **Development Engine:** Unity (C#) was chosen as our primary engine because it is the industry standard for handling high-performance 2D game loops and physics calculations.
- **Art & Assets:** We utilised Aseprite as our precision tool for creating custom 2D pixel art assets and animations, supplemented by consistent, professional visuals sourced from Kenney.nl.
- **Project Management:** We used Git & GitHub for version control (employing a feature-branch workflow with peer reviews), and Kanban boards for agile sprint tracking to maintain transparent progress.

7. Software Engineering Rigour

To keep our codebase scalable and maintainable, we applied several established design patterns and strict encapsulation:

- **State Pattern:** Applied to manage overall game phases (Menu, Playing, Paused) as well as the distinct behavioural states of individual units on the battlefield.
- **Observer Pattern:** Used extensively to decouple the UI elements (such as health bars managed by HealthBarUI and resource counters) from the underlying core game logic (managed by HealthSystem), ensuring that visual updates do not bottleneck game performance.

- **OOP Excellence & Encapsulation:** We enforced strict object-oriented programming principles across all scripts. For example, critical tracking variables like `spawnOrder` are strictly encapsulated with the `[SerializeField]` attribute, making them inaccessible to other scripts while still visible in the Unity Inspector for debugging.

8. Methodology & Quality Assurance

Consistent iterative development and testing were paramount to our success.

- **Agile Sprints:** We operated in two-week development cycles, enabling continuous, iterative feedback and rapid pivoting when mechanics required rebalancing.
- **CI/CD Workflow:** We strictly enforced a feature-branching strategy with mandatory Pull Requests (PRs), which effectively prevented regressions from breaking our main branch.
- **Testing Strategy:** We dedicated significant time to extensive bug testing and code refactoring, resulting in a highly polished user experience.

9. Conclusion for Milestone 1

Currently, our Milestone 1 stage successfully establishes the project's technical foundation through a functional combat prototype and modular gameplay systems. The current implementation demonstrates the project's feasibility and provides a scalable architecture for future gameplay extensions, such as AI progression systems, economy management, and special abilities.

10. Source Code Documentation & Engineering Notes

Currently, the GitHub repository contains detailed inline code comments and engineering notes explaining key gameplay systems, implementation decisions, and debugging considerations throughout development.

These comments were added to improve:

- Maintainability
- Scalability
- Readability
- Debugging efficiency

Key systems documented include:

- Combat and collision systems

- Movement state handling
- Raycast detection logic
- Spawn management systems
- HP synchronisation systems

As for where the relevant scripts covered in this document can be found, they are under the “Assets/Scripts/” folder.